

4. 检索流程

OpenViking 的检索并非简单向量搜索，而是结合会话上下文、检索意图、知识类型、层级结构和重排策略的分层召回流程。上层应用可按场景选择 `find` 或 `search`：`find` 适合低延迟单次查询，`search` 适合结合会话语义的复杂任务。

接口	是否使用会话	是否做意图分析	适用场景
<code>find(query)</code>	否	否	直接在指定 <code>target_uri</code> 或全局空间中做语义查找，适合搜索框、引用定位、快速问答
<code>search(query, session_id)</code>	可选	是	读取会话摘要和最近消息，生成多条 <code>TypedQuery</code> ，适合多轮对话和复杂业务任务
<code>grep(pattern)</code>	否	否	基于文本模式查找原文，适合精确关键字、编号、配置项
<code>glob(pattern)</code>	否	否	基于路径模式查找文件或目录，适合结构浏览和批量处理

4.1 search 的意图分析

`SearchService` 在存在 `session` 时会调用 `session.get_context_for_search(query)`，将压缩后的会话摘要、最近消息和当前问题交给 `IntentAnalyzer`。`IntentAnalyzer` 输出 0 到 5 条 `TypedQuery`，每条包含 `query`、`context_type`、`intent` 和 `priority`。这样一个用户问题可以被拆成资源查询、记忆查询和技能查询。

用户问题：“帮我准备某酒店集团的智能投标方案”

`IntentAnalyzer` 可能拆解为：

1. RESOURCE: 查找产品能力、技术方案、资质材料
2. MEMORY: 查找该客户或同类客户历史偏好
3. MEMORY: 查找历史投标成功案例与失败风险
4. SKILL: 查找可用的标书生成或合规检查技能

4.2 分层召回

`TypedQuery`

- > 根据 `context_type` 选择根目录
 - RESOURCE: `viking://resources`
 - MEMORY: `viking://user/memories`, `viking://agent/memories`
 - SKILL: `viking://agent/skills`
- > 全局向量召回 L0/L1 候选
- > 合并起点并按 `score / priority` 排序
- > 进入目录层级，读取 `abstract / overview`
- > 递归扩展相关子节点和 `related_uri`
- > Rerank 候选
- > 返回 `MatchedContext`